

实验六跨平台综合项目

封面信息

项目	内容
实验名称	跨平台综合项目实战
实验编号	d20301035106、d20301009206
学号	2023210649
姓名	赵翔
班级	软件 231
日期	2026-05-15
组别	第 2 组
项目名称	多端温控系统
负责技术栈	HarmonyOS 开发

实验目的

- 1. 掌握多技术栈开发能力
- 2. 体验 Git 团队协作流程
- 3. 学会 AI 辅助开发
- 4. 完成技术选型分析

实验环境

技术栈	开发工具	语言	主要依赖
Android	Android Studio	Kotlin	Android SDK
HarmonyOS	DevEco Studio	ArkTS	HarmonyOS SDK
Uniapp	HBuilderX	Vue.js	uni.request
HarmonyOS	DevEco Studio	ArkTS	HarmonyOS SDK
MAUI	Visual Studio	C#	.NET 8.0
微信小程序	微信开发者工具	JavaScript	-

第一部分：项目规划与需求分析

1. 项目选题

- 智慧健康（运动监测）
- 智慧家居（设备控制）—— 本项目选择
- 智慧校园（校园服务）

项目名称：多端温控系统（Multi-Terminal Temperature Control System）

项目背景：
随着智能家居的普及，温控系统成为家庭智能化的重要组成部分。本项目旨在开发一款支持多平台的温控系统应用，用户可通过手机、平板、小程序等多种终端实时监控室内温度，远程控制空调、地暖等设备，实现智能化温度管理。

2. 功能需求

功能模块	具体需求	技术实现
用户登录	学号密码登录，记住密码	各平台本地存储
温度监控	多房间温度实时展示，异常预警	模拟数据+定时刷新
设备控制	空调/地暖/风扇远程控制	开关状态管理
数据管理	历史数据查询与趋势分析	本地缓存+图表展示
系统设置	温度阈值、主题切换、通知	偏好设置持久化

非功能需求：

需求类型	要求	指标
性能要求	页面加载速度	< 2 秒
性能要求	温度数据刷新闻隔	5 秒
兼容性要求	支持 6 种平台	Android/HarmonyOS/Uniapp/Flutter/小程序/MAUI
安全性要求	用户数据加密存储	密码不可明文存储
可靠性要求	离线数据可查看	本地缓存最近 7 天数据

3. 团队分工

成员	技术栈	负责模块
叶庭东	Android (Kotlin + Android SDK)	登录、温控主页
赵翔	HarmonyOS (ArkTS + HarmonyOS SDK)	登录、温控主页
张天慈	Uniapp (Vue.js + CSS)	登录、温控主页
邢兆丰	Flutter (Dart + Flutter)	登录、温控主页
席永杰	微信小程序 (JS + WeChat MiniProgram)	登录、温控主页
周乐	MAUI (C# + .NET + MAUI)	登录、温控主页

4. 技术选型对比

技术栈	开发语言	UI 框架	适用平台	学习成本	性能
Android	Kotlin	XML/Jetpack Compose	Android	中等	★★★★★
HarmonyOS	ArkTS	ArkUI	HarmonyOS	中等	★★★★★
Uniapp	Vue.js	Vue 模板	全平台	低	★★★★☆☆
Flutter	Dart	Widget	全平台	中等	★★★★★
微信小程序	JavaScript	WXML/WXSS	微信	低	★★★★☆☆
MAUI	C#	XAML	全平台	中等	★★★★☆☆

5. UI 设计规范

色彩规范:

- 主色调: #1976D2 (蓝色)
- 辅助色: #42A5F5 (浅蓝)
- 警告色: #F44336 (红色, 温度异常)
- 成功色: #4CAF50 (绿色, 设备正常)
- 背景色: #F5F5F5 (浅色模式) / #1A1A2E (深色模式)

字体规范:

- 标题字号: 24sp / 18sp
- 正文字号: 16sp / 14sp
- 辅助字号: 12sp

布局规范:

- 页面边距: 16dp
- 卡片圆角: 12dp
- 卡片阴影: elevation 4
- 元素间距: 8dp / 16dp

6. 统一数据模型

用户模型 (User)

```
{  "userId": "2023210639",  "password": "210639",  "role": "admin",  "loginTime": "2026-05-03 10:00:00"}
```

房间模型 (Room)

```
{ "roomId": 1, "roomName": "客厅", "currentTemp": 24.5, "targetTemp": 25.0, "humidity": 55, "icon": "home", "isAlert": false}
```

设备模型 (Device)

```
{ "deviceId": 1, "deviceName": "客厅空调", "deviceType": "air_conditioner", "roomId": 1, "isOn": true, "temperature": 25, "mode": "cool", "speed": "medium"}
```

温度记录模型 (TempRecord)

```
{ "recordId": 1, "roomId": 1, "temperature": 24.5, "humidity": 55, "recordTime": "2026-05-03 10:00:00"}
```

第二部分：Git 团队协作

1. Git 仓库创建

```
# 创建项目仓库 git init multi-temp-controlled multi-temp-control
# 创建主分支 git checkout -b main
# 创建开发分支 git checkout -b develop
# 创建各技术栈功能分支 git checkout -b feature/android git checkout -b feature/harmonyos git checkout -b feature/uniapp git checkout -b feature/harmonyos-2 git checkout -b feature/wechat-miniprogram git checkout -b feature/maui
```

2. .gitignore 配置

```
# 通用忽略.DS_StoreThumbs.db*.log*.tmp.env
# Node.jsnode_modules/npm-debug.log*
# Android.gradle/build/*.apk*.aablocal.properties.idea/
# HarmonyOS.hvigor/oh_modules/build/entry/build/
# Uniappunpackage/.hbuilderx/
# MAUIbin/obj/*.user*.suo
# 微信小程序miniprogram_npm/
# IDE.vscode/.idea/*.swp*.swo
```

3. 代码提交规范

- 提交信息格式: feat/fix/docs/style/refactor/test: 具体修改内容
- 定期拉取最新代码: git pull origin develop
- 提交前进行代码审查
- 完成后提交 Pull Request 到 develop 分支
- 代码审核通过后合并

4. 项目结构

```

multi-temp-control/ |—— android/                # Android 项目 |—— harmonyos/
# HarmonyOS 项目 |—— uniapp/                      # Uniapp 项目 |—— flutter/
# Flutter 项目 |—— wechat-miniprogram/          # 微信小程序项目 |—— maui/
# MAUI 项目 |—— docs/                          # 文档 |—— requirements.md    # 需求
文档 |—— design.md                            # 设计文档 |—— api.md            # 接口文
档 |—— README.md                             # 项目说明

```

5. 各技术栈项目初始化

Android 项目：使用 Android Studio 创建，语言 Kotlin，最低 SDK API 24

HarmonyOS 项目（赵翔）：使用 DevEco Studio 创建，语言 ArkTS，SDK HarmonyOS 5.0+

Uniapp 项目：使用 HBuilderX 创建，Vue 3 版本

HarmonyOS 项目（邢兆丰）：使用 DevEco Studio 创建，语言 ArkTS，SDK HarmonyOS 5.0+

微信小程序项目：使用微信开发者工具创建，使用测试号

MAUI 项目：使用 Visual Studio 创建，框架.NET 8.0

6. AI 辅助开发环境配置

各成员使用 TRAE AI 编程助手进行开发，配置项目规则：

```

# 多端温控系统开发规范
## 代码风格- 使用 2 个空格缩进- 变量和函数使用驼峰命名- 类名使用大驼峰命名- 常量使用全大写下划线分隔
## 项目约定- 温度单位默认摄氏度- 温度预警阈值为 30° C- 数据缓存有效期 30 分钟- 模拟数据更新间隔 5 秒
## UI 规范- 主色调：#1976D2- 页面边距：16dp- 卡片圆角：12dp- 字号：标题 24sp / 正文 16sp / 辅助 12sp

```

第三部分：多端实现

1. Android 开发（叶庭东）

采用 Kotlin 语言开发，实现登录和温控主页功能，包含温度数据展示和模拟更新逻辑。

2. HarmonyOS 开发（赵翔）

2.1 核心代码

```
// pages/Index.ets@Entry@Componentstruct Index { @State rooms: Room[] =
[ { roomId: 1, roomName: '客厅', currentTemp: 24.5, targetTemp: 25.0,
humidity: 55 }, { roomId: 2, roomName: '卧室', currentTemp: 22.3,
targetTemp: 23.0, humidity: 50 }, { roomId: 3, roomName: '厨房',
currentTemp: 26.8, targetTemp: 25.0, humidity: 60 }, { roomId: 4, roomName:
'卫生间', currentTemp: 23.1, targetTemp: 24.0, humidity: 65 } ] build()
{ Column() { Text('多端温控系统
') .fontSize(24) .fontWeight(FontWeight.Bold) .margin({
top: 20, bottom: 20 }) ForEach(this.rooms, (room: Room) =>
{ Row() { Text(room.roomName).fontSize(18).layoutWeight(1)
Text(`${room.currentTemp}°
C`).fontSize(20).fontColor('#1976D2') } .width('90%') .p
adding(16) .backgroundColor(Color.White) .borderRadius(12)
.margin({ bottom:
8 }) }) } .width('100%') .height('100%') .backgroundColor('#
F5F5F5') }}
```

2.2 登录页面

```
// pages/Login.ets@Entry@Componentstruct Login { @State userId: string = ''
@State password: string = '' @State rememberMe: boolean = false build()
{ Column()
{ Image($r('app.media.logo')) .width(100) .height(100)
.margin({ top: 80, bottom: 30 }) Text('多端温控系统
') .fontSize(28) .fontWeight(FontWeight.Bold) .margin({
bottom: 50 }) TextInput({ placeholder: '请输入学号
' }) .width('80%') .height(48) .backgroundColor(Color.Wh
ite) .borderRadius(8) .padding({ left:
16 }) .margin({ bottom: 16 }) .onChange((value: string) =>
{ this.userId = value }) TextInput({ placeholder:
'请输入密码
' }) .type(InputType.Password) .width('80%') .height(48)
.backgroundColor(Color.White) .borderRadius(8) .padding
({ left: 16 }) .margin({ bottom: 16 }) .onChange((value: string)
=> { this.password = value }) Row()
{ Checkbox() .select(this.rememberMe) .onChange((isS
elect: boolean) => { this.rememberMe = isSelect })
Text('记住密码') .fontSize(14) .margin({ left:
8 }) } .width('80%') .margin({ bottom: 30 })
Button('登录
') .width('80%') .height(48) .backgroundColor('#1976D2')
.fontColor(Color.White) .borderRadius(8) .onClick(() =>
{ this.handleLogin() }) } .width('100%') .height('10
0%') .backgroundColor('#F5F5F5') } handleLogin() { if (this.userId
&& this.password) { if (this.rememberMe) { // 保存登录信息
localStorage.setItem('user', JSON.stringify({ userId: this.userId,
password: this.password })) } router.pushUrl({ url:
'/pages/Index' }) } }}
```

3. Uniapp 开发（张天慈）

采用 Vue.js 框架开发，实现登录和温控主页功能，支持温度数据实时更新展示。

4. HarmonyOS 开发（邢兆丰）

采用 ArkTS 语言开发，实现登录和温控主页功能，与赵翔负责的模块功能一致。

5. 微信小程序开发（席永杰）

采用 JavaScript 语言开发，实现登录和温控主页功能，通过微信小程序框架 API 实现温度监控。

6. MAUI 开发（周乐）

采用 C#语言和.NET 框架开发，实现登录和温控主页功能，支持跨平台部署。

第四部分：测试验证与打包部署

1. 测试用例

测试项	预期结果
应用启动	正常启动
温度数据展示	实时更新
页面切换	流畅无卡顿
数据存储	数据持久化
设备控制	开关状态正确
温度预警	超阈值红色标识

2. 打包部署

平台	打包命令	部署方式
Android	./gradlew assembleRelease	APK 安装
HarmonyOS	DevEco Studio 构建	HAP 安装
Flutter	flutter build apk --release	APK 安装
Uniapp	HBuilderX 云打包	多端部署
MAUI	dotnet publish -f net8.0-android	APK 安装
微信小程序	上传体验版	二维码扫描

第五部分：技术选型对比报告

1. 开发效率对比

技术栈	开发用时	代码量	难度评分
Android	4 小时	~350 行	3/5
HarmonyOS	5 小时	~400 行	4/5
Uniapp	2 小时	~200 行	2/5
Flutter	3 小时	~300 行	3/5
MAUI	4 小时	~350 行	3/5
微信小程序	2 小时	~180 行	2/5

2. 性能对比

技术栈	启动速度	内存占用	流畅度
Android	<1 秒	~80MB	★★★★★
HarmonyOS	<1 秒	~90MB	★★★★★
Uniapp	1-2 秒	~120MB	★★★★☆☆
Flutter	<1 秒	~100MB	★★★★★
MAUI	1-2 秒	~110MB	★★★★☆☆
微信小程序	1-2 秒	~70MB	★★★★☆☆

3. 适用场景分析

技术栈	适用场景	优势	劣势
Android	原生应用开发	性能最佳，生态成熟	仅限 Android 平台
HarmonyOS	鸿蒙原生开发	分布式能力，多端适配	生态尚在发展
Uniapp	多端快速开发	开发效率高，一套代码多端	性能一般，复杂功能受限
Flutter	跨平台开发	性能接近原生，UI 一致性好	学习成本中等，包体积大
MAUI	.NET 生态开发	C#生态，MVVM 成熟	生态相对薄弱
微信小程序	轻量级应用	用户基数大，无需安装	功能受限，依赖微信

第六部分：项目总结报告

1. 项目概述

- 项目名称：多端温控系统
- 开发目标：实现多平台温度监控与设备控制
- 团队成员：6 人

2. 开发过程

- 需求分析：确定功能需求和技术选型
- 技术选型：选择 6 个主流技术栈
- 开发实现：各成员独立开发
- 测试上线：功能测试和兼容性测试

3. 收获与体会

- 技术成长：掌握了多技术栈开发
- 团队协作：体验了 Git 团队协作流程
- AI 工具使用：学会了使用 TRAE AI 辅助开发

4. 问题与改进

- 遇到的问题：HarmonyOS 开发环境配置复杂，ArkTS 语法需要适应
- 解决方案：通过团队协作和 TRAE AI 辅助克服技术难点
- 改进方向：加强 HarmonyOS 生态学习，优化性能

实验结果

验证结果

技术栈	应用运行	功能完整	性能表现
Android	[] 成功	[] 成功	[] 良好
HarmonyOS	[] 成功	[] 成功	[] 良好
Uniapp	[] 成功	[] 成功	[] 良好
Flutter	[] 成功	[] 成功	[] 良好
MAUI	[] 成功	[] 成功	[] 良好
微信小程序	[] 成功	[] 成功	[] 良好

截图记录

HarmonyOS 运行截图



图 1 HarmonyOS 温控系统主页



图 2 HarmonyOS 登录页面

考核指标

考核项	评分标准	得分
功能完整性	各技术栈应用功能完整	20 分
代码质量	代码结构清晰，注释完整	15 分
团队协作	Git 流程规范，协作良好	20 分
多端实现	至少完成 3 个技术栈的开发	25 分
技术对比	完成技术选型分析报告	10 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
团队协作	
创新点	
综合评价	
成绩	

教师签名： _____

日期： _____